

基于 Monaco 的 AI 增强型集成开发环境的设计与实现

综合实践项目团队组建报告

创建新组——夏盛宇&王家栋&彭施博&曹宇轩

一、选题目的和意义

随着人工智能技术的快速发展，大语言模型（LLM）在软件开发中的应用越来越广泛。传统的集成开发环境（IDE）主要提供代码编辑、编译和调试等基本功能，而在智能化辅助方面仍存在一定的不足。近年来，诸如 GitHub Copilot 等 AI 编程助手逐渐成为开发者的重要工具，因此研究并开发一种融合人工智能技术的智能开发环境具有重要意义。

本项目旨在设计并实现一个基于 Monaco Editor 的 AI 增强型集成开发环境（AI-Enhanced IDE）。系统通过结合现代前端技术与人工智能技术，为开发者提供代码编辑、文件管理、终端执行、Git 管理以及 AI 辅助编程等功能。项目将以 Electron 桌面框架构建跨平台应用界面，并采用 C++ 语言实现部分核心服务模块，例如文件系统管理、命令执行与代码分析，以提高系统性能和稳定性。通过本项目的研究与开发，可以加深对现代软件开发工具架构的理解，提升团队成员在 C++ 编程、前端开发、系统架构设计以及人工智能接口调用等方面的综合能力，同时探索 AI 技术在软件开发工具中的实际应用模式，为未来智能开发环境的研究提供实践基础。

二、研究内容、研究方法、技术路线及可行性分析

（一）研究内容

本项目主要设计并实现一个 AI 增强型集成开发环境，其主要功能模块包括以下几个方面。

1. 代码编辑模块

系统采用 Monaco Editor 作为代码编辑器核心，实现完整的代码编辑功能，包括语法高亮、自动语言识别、多标签页编辑、代码折叠以及基本的代码补全提示等功能，使开发者能够在 IDE 中高效编写代码。

2. 文件管理模块

系统实现类似于常见 IDE 的文件资源管理器，支持项目目录浏览以及文件操作，包括文件和文件夹的创建、删除、重命名、打开与保存等操作，方便开发者管理项目结构。

3. 集成终端模块

系统内置终端模拟功能，使开发者可以直接在 IDE 中执行命令行操作，例如编译程序、运行脚本以及执行系统命令等。终端支持多标签页运行，方便开发者同时执行多个任务。

4. AI Agent 智能辅助模块

系统集成 AI Agent 功能，通过调用大语言模型 API，为开发者提供智能编程辅

助服务。例如自动生成代码、解释代码逻辑、修改代码内容以及回答编程问题等。同时 AI Agent 还可以通过工具调用实现读取文件、写入文件、搜索代码以及执行终端命令等功能，从而实现更高层次的代码辅助。

5. 代码搜索与项目分析模块

系统提供全局搜索功能，支持根据文件名或文件内容进行快速搜索，同时可以对项目代码结构进行分析，提高开发者查找代码和理解项目结构的效率。

6. Git 版本管理模块

系统集成 Git 版本控制功能，开发者可以在 IDE 内直接查看仓库状态、暂存与提交代码、创建或切换分支以及查看代码差异（Diff），从而提高开发效率。

（二）研究方法

本项目采用模块化设计与分层架构方法进行开发，整体系统主要分为前端界面层、通信层以及系统服务层。

首先，在前端界面层使用现代 Web 技术构建用户界面，例如 React 或 Vue 框架结合 TypeScript 语言开发，并利用 Tailwind CSS 实现界面样式设计，从而提高界面开发效率与可维护性。

其次，在通信层通过 Electron 提供的 IPC（进程间通信）机制实现前端界面与系统服务之间的数据交互，使用户操作能够传递到后端模块进行处理，并将处理结果返回到界面进行显示。

最后，在系统服务层使用 C++ 语言实现核心功能模块，例如文件系统管理、终端命令执行、代码搜索以及 Git 操作接口等。通过 Node 原生扩展或 WebAssembly 技术将 C++ 模块与 Electron 应用进行连接，从而兼顾系统性能与开发灵活性。

在人工智能功能方面，系统通过调用多种大语言模型 API，例如 OpenAI、DeepSeek、通义千问以及本地模型服务，实现 AI 编程辅助能力。

（三）技术路线

本项目整体系统架构主要包括用户界面层、IDE 服务层以及 AI 服务层。用户界面层负责系统界面的展示与交互，由 React 或 Vue 等现代前端框架实现。IDE 服务层主要由 C++ 实现，负责文件管理、终端管理、代码搜索以及 Git 操作等核心功能。AI 服务层则通过调用大语言模型 API，实现智能代码辅助功能。各模块之间通过 Electron 的 IPC 通信机制进行数据交换，从而构建一个完整的 AI 增强型开发环境。

（四）可行性分析

在技术可行性方面，本项目所采用的关键技术均为当前成熟并广泛应用的技术。例如 Monaco Editor 是 Visual Studio Code 所使用的编辑器核心，Electron 是成熟的跨平台桌面应用开发框架，而 C++ 语言则具有良好的性能和系统开发能力。此外，大语言模型 API 已经在多种 AI 编程助手中得到广泛应用，因此技术实现具有较高可行性。

在实现难度方面，本项目可以按照模块逐步开发，例如先实现基础的代码编辑与文件管理功能，再逐步加入终端模块、Git 管理以及 AI Agent 功能。通过阶段性开发与测试，可以有效降低开发难度。

在开发环境方面，本项目仅需要常见的软件开发工具，例如 Node.js、C++ 编译环境以及现代前端开发工具，这些工具均易于获取与配置，因此项目实施具有较好的环境支持。

三、项目特色与创新点

1. AI Agent 深度集成 IDE

本项目不仅提供简单的 AI 聊天功能，而是将 AI Agent 深度集成到 IDE 中，使其能够读取项目代码、修改文件内容以及执行终端命令，从而实现更加智能化的编程辅助。

2. C++ 高性能核心模块

传统 Electron IDE 多采用 JavaScript 实现系统功能，而本项目将核心系统服务使用 C++ 实现，例如文件管理与代码搜索模块，以提高系统性能并减少资源占用。

3. 多模型 AI 支持

系统支持多种大语言模型服务，例如 OpenAI、DeepSeek、通义千问以及本地部署模型，用户可以根据需求自由选择不同的 AI 服务。

4. 模块化 IDE 架构设计

项目采用模块化架构设计，将编辑器模块、AI 模块、Git 模块以及搜索模块进行独立设计，便于未来扩展插件系统和新功能。

四、进度安排

第 1 周：完成项目需求分析与课题调研。

第 2 周：完成系统架构设计并搭建开发环境。

第 3—4 周：实现代码编辑器模块与文件管理模块。

第 5 周：实现终端模块并完成基本运行功能。

第 6 周：实现 Git 集成功能。

第 7 周：实现 AI Agent 模块并接入大语言模型接口。

第 8 周：进行系统测试与功能优化。

第 9 周：整理项目文档并完成项目报告。

五、成员名单、各成员角色及任务分配

本团队由四名计算机专业学生组成，采用 Scrum 敏捷开发模式进行项目管理。团队内部根据系统架构的层次性（表示层、业务逻辑层、底层服务层）进行了明确的分工，确保每位成员既有核心的开发任务，又承担相应的软件工程管理职责：

1. 曹宇轩：Scrum Master 与前端架构师

作为团队的 Scrum Master，曹宇轩负责主持每日站会，监控项目 14 周的进度，并协调前后端接口的对接规范。在开发任务上，他主要负责基于 Electron 的主窗口架构开发，以及 Monaco Editor 内核的深度定制。这包括实现多标签页管理系统、自定义语法高亮、代码 Diff 视图对比，以及编辑器与底层 IPC 通信机制的建立，确保前端交互的高性能与流畅度。

2. 彭施博：系统后端与底层服务开发

负责项目最核心的“硬核”部分，即基于 C++ 的核心服务层开发。他主要利用 C++ 实现高性能的文件系统监控、工程级全局搜索、嵌入式终端命令解析以及本地 Git 逻辑的封装。同时，他担任系统架构师角色，负责撰写系统架构设计说明书，并维护项目的技术拓扑图，确保底层服务与上层 Electron 应用的稳定连接。

3. 王家栋：AI 引擎算法与产品负责人 (PO)

担任 Product Owner，负责定义产品的功能优先级，维护 Product Backlog（功

能清单)，并撰写需求规格说明书。在技术实现上，他专注于 AI 引擎与上下文管理逻辑。他需要实现基于 RAG（检索增强生成）的代码上下文提取算法，设计 AI Agent 的对话流逻辑，并对接 OpenAI、DeepSeek 等大模型 API，实现真正的“沉浸式”AI 代码修改功能。

4. 夏盛宇：测试、运维与质量保障工程师

负责整个软件生命周期的质量控制与部署。他需要编写详尽的单元测试与集成测试脚本，负责 Electron 应用在不同环境下的打包、分发以及在希冀平台上的部署与环境配置。此外，他还负责维护 Git 仓库的代码规范，撰写项目质量监控报告，并根据团队成员的提交记录汇总项目进度产物。